IBM APPLIED DATA SCIENCE CAPSTONE

SpaceX Falcon 9
First-Stage Landing Prediction

Data collection • EDA • SQL • Geospatial analytics • Classification

Prepared by: Sanskruti Salve

https://github.com/Sanskruti104/IBM-SpaceX-Data-Science-Capstone

DRAFT: benchmark/reference content must be verified against your executed notebooks.

Executive Summary

- Problem: estimate first-stage landing success from historical launch characteristics.
- Methods: SpaceX API + web scraping; data wrangling; EDA/SQL; Folium and Dash; four classifiers.
- Public IBM capstone examples report similar model performance on a small test set. Reproduce metrics in your own repository before final submission.

Introduction & Problem Statement

- Reuse of Falcon 9 first stages can reduce launch costs.
- Question: can flight number, payload, orbit, and launch site help classify landing outcomes?
- Target: Class = 1 for recorded successful landing; 0 for a known unsuccessful attempt. Keep unknown outcomes separate.
- Educational decision support only; not an engineering or safety guarantee.

End-to-End Methodology

- Collect API + web data → clean and label → explore with EDA and SQL → map and dashboard → train and evaluate classifiers.
- Keep data sources and retrieval dates documented.
- Use course data snapshot to align with the graded IBM labs; live API data can change.

Data Collection — SpaceX API

- Request launch records → parse JSON → normalize nested fields → filter Falcon 9 → save snapshot.
- Collect launch date, flight number, payload, launch site, core landing attempt, and landing result where available.
- Notebook: notebooks/01_data_collection_api.ipynb

Data Collection — Web Scraping

- Request the launch-history page → inspect HTML tables → parse the relevant table → normalize columns → validate and save.
- Cross-check launch names, dates, and outcomes against API records.
- Notebook: notebooks/02_web_scraping.ipynb

Data Wrangling Methodology

- Inspect types and missingness; remove duplicates; standardize categories; parse dates and numeric fields.
- Construct target only from known landing attempts and known outcomes.
- Use one-hot encoding and scaling in a pipeline; fit preprocessing on training data only.
- Notebook: notebooks/03_data_wrangling.ipynb

EDA & Data Visualization Methodology

- Flight number vs outcome: explore whether outcomes vary across launch sequence.
- Payload vs outcome: compare payload ranges for successes and failures.
- Launch site/orbit: compare counts and rates; note denominators.
- Yearly trend: inspect changes over time; show counts alongside rates.
- Every plot needs labels, legend, sample size, and a data-supported takeaway.

EDA Results — Relationships

- Course examples commonly report higher success in later flights; validate against your plot.
- KSC LC-39A is often reported as having a high success rate in the historical snapshot; compare both counts and rates.
- Rare orbits may show very high rates based on few records.
- Payload effects may interact with orbit, site, and booster configuration. These are associations, not causes.

EDA Results — Required Charts

- Include actual scatter plots: FlightNumber vs Class; PayloadMass vs Class.
- Include bar charts: success by launch site and orbit.
- Include yearly success trend line.
- Replace coverage schematic with executed notebook charts before submission.

EDA with SQL — Queries

- Launch sites: SELECT DISTINCT LaunchSite FROM launches;
- Payload: SELECT AVG(PayloadMass), MAX(PayloadMass) FROM launches;
- Outcome counts: SELECT Outcome, COUNT(*) FROM launches GROUP BY Outcome;
- Yearly activity: SELECT year, COUNT(*) FROM launches GROUP BY year ORDER BY year;
- Also answer NASA CRS payload, F9 v1.1 average payload, first successful ground-pad landing, and maximum-payload booster version.

EDA with SQL — Results

- Show executed query and result table for each question.
- Rank launch sites using counts and success proportions; report denominators.
- Summarize payload statistics and time-based patterns.
- Do not submit query descriptions without actual output tables.

Folium Map

- Load launchpad coordinates → create base map → add markers/popups → cluster records → export HTML.
- Popups should show site and launch summaries.
- For proximity analysis, calculate distances to sourced geographic features; map appearance alone does not prove proximity.
- Notebook: notebooks/06_folium_map.ipynb

Plotly Dash

- Required: success/failure pie chart and payload-vs-outcome scatter plot.
- Add launch-site dropdown and make charts respond to selection.
- Capture screenshots of the running app and a filtered state.
- App code: dashboard/app.py

Predictive Analysis — Classification

- Models: Logistic Regression, SVM, Decision Tree, KNN.
- Split training/test data; tune hyperparameters with cross-validation on training data only.
- Report accuracy, precision, recall, F1-score, and confusion matrix.
- Notebook: notebooks/07_predictive_analysis.ipynb

Predictive Analysis — Model Results

- Reference baseline from a public IBM capstone example: 83.33% test accuracy for each of four models on an 18-record test set.
- Small test sets make accuracy unstable; one changed prediction materially affects the score.
- Replace reference values with the model comparison generated from your own executed notebook.

Confusion Matrix

- True positive: successful landing correctly predicted.
- True negative: unsuccessful landing correctly predicted.
- False positive: model predicts landing success when it fails—potentially costly for cost estimates.
- False negative: model predicts failure when landing succeeds.
- Insert the numeric matrix from your own model output.

Innovative Insights

- Show sample counts with success rates; rare orbit categories can appear perfect with very few observations.
- Consider time-based validation to test generalization to later launches.
- Discuss false-positive and false-negative costs, not just accuracy.
- These are recommendations; empirical claims must be checked against actual outputs.

Conclusion

- The project links data collection, cleaning, EDA, SQL, geospatial mapping, dashboards, and classification.
- Associations in historical data do not prove causation.
- Choose the model using cross-validation, test metrics, interpretability, and error costs.
- Limitations: small sample, missing values, evolving hardware, mission profiles, and temporal drift.
- Update the conclusion after reproducing the notebook results.

Appendix — Repository & Checklist

- Repository: https://github.com/Sanskruti104/IBM-SpaceX-Data-Science-Capstone
- Notebook order: API → scraping → wrangling → EDA → SQL → Folium → classification.
- Include requirements, notebooks, outputs, map HTML, dashboard, report PDF, and presentation PDF.
- References: IBM course labs; SpaceX API; Wikipedia launch history; public IBM capstone benchmark cited as context only.
- Final: remove placeholders and include actual output screenshots/tables.